

Introduction to Neural Networks

Alexander McLain

September 24, 2026

Why Neural Networks?

- ▶ Neural networks (NNs) are inspired by how neurons in the brain process signals.
- ▶ Each “neuron” takes inputs, applies a weight, adds a bias, and passes the result through a non-linear activation function.
- ▶ They are extremely flexible and can model highly non-linear relationships between predictors and outcomes.
- ▶ Modern applications include voice recognition, medical image classification, and natural language processing.

Biological Inspiration

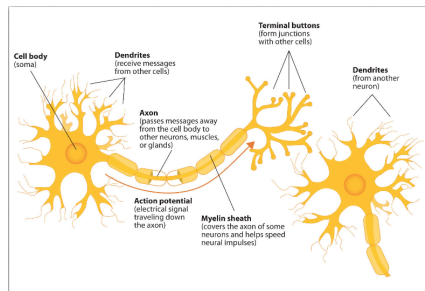


Figure 1: Artificial neuron modeled loosely after biological neurons.

- ▶ Inputs ($x_1, x_2, \dots, x_p$) represent signals from other neurons.
- ▶ Weights ($w_1, w_2, \dots, w_p$) control the strength of each input.
- ▶ The activation function determines if the neuron “fires.”

Basic Structure

- ▶ An NN with 1 layer with 1 neuron computes

$$\hat{y} = \sigma \left(b_0 + \sum_{j=1}^p w_j x_j \right)$$

where $\sigma(\cdot)$ is the activation function.

- ▶ If σ is linear, the resulting function would be linear. Without nonlinear activation functions, adding layers does not make the model nonlinear.
- ▶ Mostly σ is nonlinearity.
- ▶ This can be viewed as a flexible generalization of logistic or linear regression.
- ▶ Allows neural networks to approximate complex relationships.

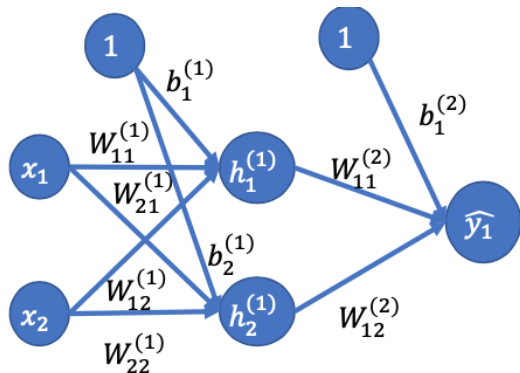
Basic Structure

- If we have 1 layer with 2 neurons with 2 variables

$$h_1^{(1)} = \sigma \left(b_1^{(2)} + W_{11}^{(1)} x_1 + W_{12}^{(1)} x_2 \right)$$

$$h_2^{(1)} = \sigma \left(b_2^{(2)} + W_{21}^{(1)} x_1 + W_{22}^{(1)} x_2 \right)$$

$$\hat{y} = \sigma \left(b_1^{(2)} + W_{11}^{(2)} h_1^{(1)} + W_{12}^{(2)} h_2^{(1)} \right)$$



Basic Structure

- ▶ We can have multiple layers with a varying number of neurons per layer.
- ▶ If we have 1 layer with K neurons

$$\hat{y} = \sigma \left(b_k^{(1)} + \sum_{k=1}^K W_{1k}^{(2)} h_k^{(1)} \right)$$

where $h_k^{(1)} = \sigma \left(b_1^{(2)} + \sum_{j=1}^p W_{1k}^{(1)} x_j \right)$.

Basic Structure

- ▶ We can have multiple layers with a varying number of neurons per layer.
- ▶ If we have 1 layer with K neurons

$$\hat{y} = \sigma \left(b_k^{(1)} + \sum_{k=1}^K W_{1k}^{(2)} h_k^{(1)} \right)$$

where $h_k^{(1)} = \sigma \left(b_1^{(2)} + \sum_{j=1}^p W_{1k}^{(1)} x_j \right)$.

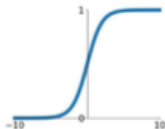
- ▶ Another way to write this is

$$\hat{y} = \sigma \left\{ b_1^{(2)} + \sum_{k=1}^K W_{1k}^{(2)} \sigma \left(b_1^{(2)} + \sum_{j=1}^p W_{1k}^{(1)} x_j \right) \right\}$$

Common Activation Functions

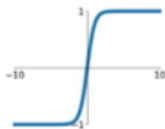
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



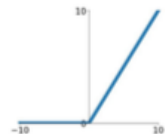
tanh

$$\tanh(x)$$



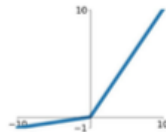
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

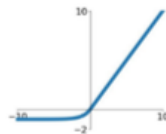


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Universal Function Approximation

- ▶ A single hidden layer with enough neurons can approximate *any* continuous function.
- ▶ This makes NNs extremely flexible but also potentially overparameterized.
- ▶ In practice, deeper (multi-layer) networks often require fewer parameters for the same approximation.

Backward Propagation

- ▶ Learning involves minimizing a loss function (e.g., MSE, cross-entropy).
- ▶ Backpropagation computes gradients efficiently using the chain rule.
- ▶ Updates weights iteratively using algorithms like stochastic gradient descent (SGD) or Adaptive Moment Estimation (Adam)

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \eta \frac{\partial L}{\partial w_{ij}}$$

where η is a learning rate.

Overview of optimization methods.

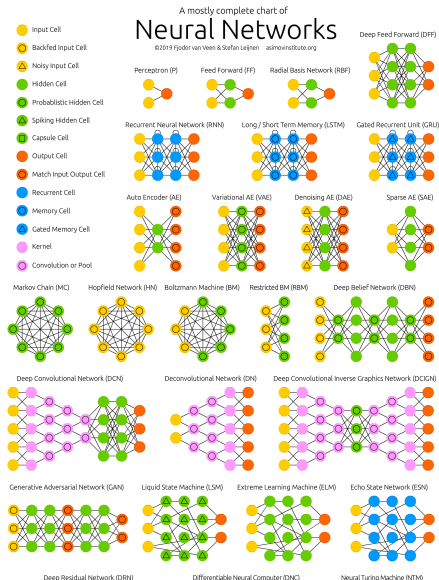
Regression vs Classification

- ▶ **Regression:** Continuous outcomes
 - ▶ Hidden layers: ReLU, etc.
 - ▶ Output: linear
 - ▶ Loss: MSE (or another appropriate regression loss)
- ▶ **Classification:** Discrete outcomes
 - ▶ Hidden layers: ReLU, etc.
 - ▶ Output: sigmoid (binary) or softmax (multiclass)
 - ▶ Loss: cross-entropy
- ▶ Neural networks unify both frameworks by changing the output layer and loss function.

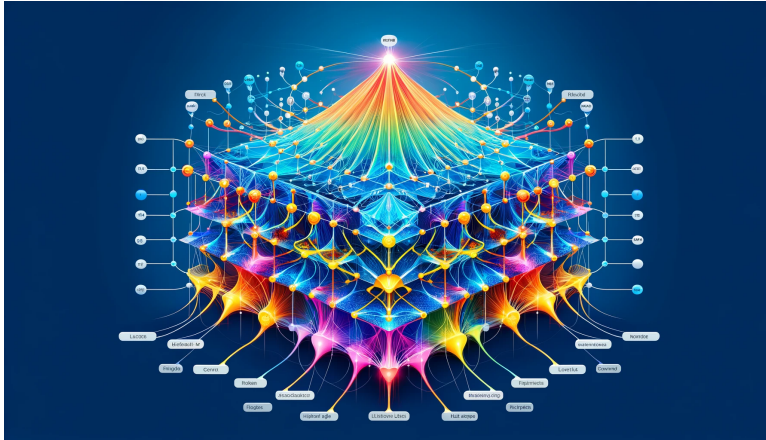
Hyperparameters in Neural Networks

Hyperparameters in Neural Networks include (and certainly aren't limited to):

- ▶ Architecture, i.e., number of hidden layers and neurons
- ▶ Activation function choice
- ▶ Optimization method (SGD, ADAM)
- ▶ Learning rate (η)
- ▶ dropout / weight decay
- ▶ batch size
- ▶ number of epochs / early stopping



Things can get really complicated

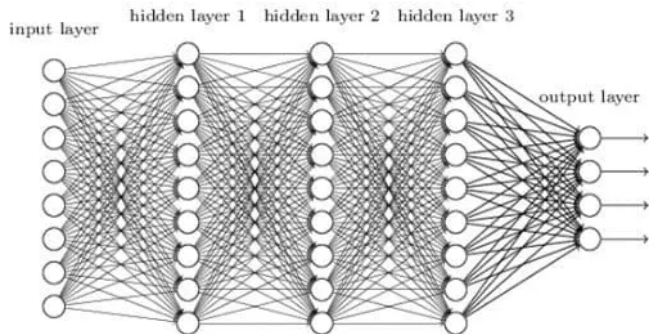


Deep Neural Networks

- ▶ “Deep” = multiple hidden layers.
- ▶ Layers learn hierarchical representations.
- ▶ Early layers learn simple patterns; deeper layers learn complex abstractions.
- ▶ Require large datasets and computational power.

Deep Neural Networks

Deep neural network



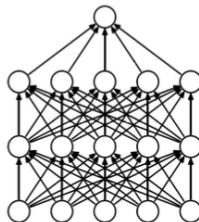
- ▶ A Deep NN has at least three hidden layers.
- ▶ Neural networks have many tuning parameters, so model development requires systematic validation and tuning.

Weight Regularization

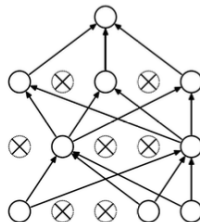
- ▶ Penalizes large weights to prevent overfitting.
- ▶ Common forms:
 - ▶ L1 (Lasso): encourages sparsity.
 - ▶ L2 (Ridge): smooths weight magnitudes.
- ▶ Often combined with dropout or early stopping.

Dropout Regularization and Batch Size

- ▶ During training, randomly set a fraction of hidden-unit outputs to zero.
- ▶ Prevents co-adaptation and forces redundancy in representations.
- ▶ Effectively averages many smaller networks.
- ▶ **Batch size** is the number of samples that will be propagated through the network.



(a) Standard Neural Net



(b) After applying dropout.

Weight Initialization

- ▶ Good initialization can improve optimization and reduce problems with vanishing or exploding gradients.
- ▶ Common strategies:
 - ▶ Xavier initialization (for sigmoid/tanh)
 - ▶ He initialization (for ReLU)
- ▶ Combine with normalization and careful learning rate tuning.

Unbalanced Datasets

- ▶ Many health outcomes are rare (e.g., disease detection).
- ▶ Imbalance can cause bias toward the majority class.
- ▶ Solutions:
 - ▶ Oversampling minority class (SMOTE)
 - ▶ Class weights in loss function
 - ▶ Anomaly detection methods

Batch Normalization

- ▶ **Batch normalization:** Normalizes activations within each layer to stabilize and speed up learning.
 - ▶ Normalizes intermediate activations to make training more stable.
 - ▶ Allows larger learning rates
 - ▶ Reduces the importance of initialization
- ▶ These enable much deeper and more stable architectures.

Tools for Neural Networks in R

- ▶ **Keras:** high-level interface; convenient, but depends on a Python/backend environment.
- ▶ **Torch:** deep-learning framework available directly from R.
- ▶ **Luz:** high-level training interface built on Torch.
- ▶ In practice, software environment and reproducibility matter alongside model performance.

Data Needs for Neural Networks

- ▶ Deep learning models perform best with large, diverse, and high-quality data.
- ▶ With small datasets, they risk overfitting and poor generalization.
- ▶ When public health data are limited, we can
 - ▶ **Data augmentation:** create new training examples from existing ones by applying small transformations that don't change their meaning (common in image or text data).
 - ▶ **Transfer learning:** start with a pretrained neural network on a much larger dataset (UK Biobank) and specialize it to your data.
 - ▶ **Simpler models:** logistic regression (including LASSO variants), random forests, or gradient boosting.

Summary: What Can Be Tuned?

- ▶ Architecture: number of layers, neurons
- ▶ Learning rate and optimizer
- ▶ Activation functions
- ▶ Regularization (dropout, L1/L2)
- ▶ Batch size and number of epochs
- ▶ Initialization strategy
- ▶ Data balancing and augmentation

Takeaways

- ▶ Neural networks are powerful, flexible tools that can approximate complex relationships.
- ▶ They require careful tuning, sufficient data, and thoughtful interpretation.
- ▶ In public health, use them when relationships are highly nonlinear or involve unstructured data (e.g., images, text, or signals).